DT081A – Signal and Image Processing

Laboratory 1 Report

Sections 1.1–1.3 and 2.1 – Signal and Image Processing

Jakob Obid

4 November 2026

1 Introduction

This part of Lab 1 compares sine, square and sawtooth waves. It shows how frequency and addition change a signal. FFTs were also used to study fixed frequencies, noise and a chirp. I also tested how different sampling rates affect a signal. I also compared image resampling and requantization.

2 Experimental Setup

NumPy and Matplotlib were used. All signals use the teacher's time vector, `t = np.linspace(0, 1, 1000)`: 1,000 points from 0 to 1 second, including both ends. The starting signals are a 10 Hz sine and a 5 Hz square. Their amplitudes were kept unchanged. For Section 1.2, the original 10 Hz and 20 Hz sines have amplitudes 1 and 0.5. The FFT uses the time step between the given points. I used the teacher's sampling examples and tested a 23 Hz sine wave sampled at 8 Hz. Pillow loaded Lenna and a grass texture as grayscale images.

3 Results and Analysis

3.1 Signal Generation and Visualization

Q1. Modify the code to create a sawtooth wave. Describe the mathematical function you used.

Answer: The function is $s(t) = 2(5t - \lfloor 5t \rfloor) - 1$. Here, $\lfloor 5t \rfloor$ means rounding down to a whole number. Subtracting it leaves the part of a cycle between 0 and 1. Scaling by 2 and subtracting 1 gives a rising ramp from -1 towards $+1$. It jumps back every 0.2 seconds, so its frequency is 5 Hz (Figure 1).

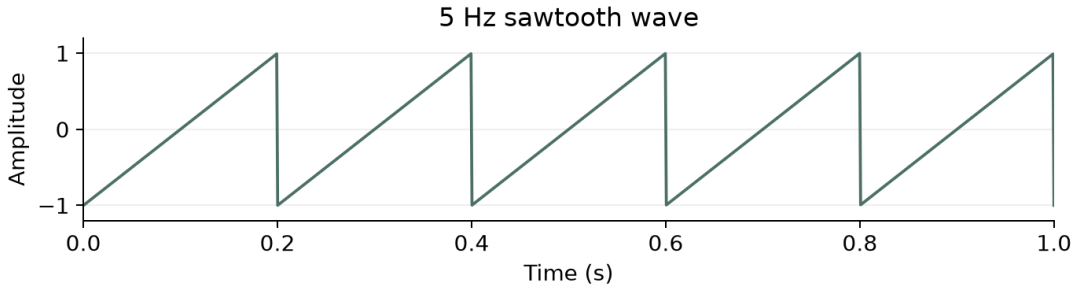


Figure 1: Five rising ramps during one second. Each ramp ends with a jump back.

Q2. Change the frequency of the sine wave to 20 Hz. How does this affect the plot?

Answer: The 20 Hz sine has 20 cycles in one second instead of 10. The waves are closer together in time. The amplitude stays the same (Figure 2).

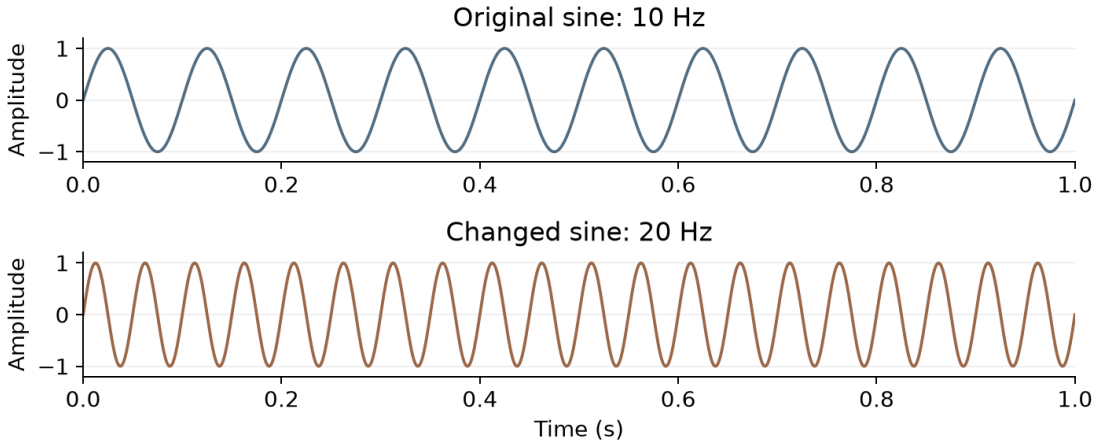


Figure 2: The same one-second interval and amplitude limits show twice as many cycles at 20 Hz.

Q3. What happens to the square wave if you change the sign function to `np.ceil(np.sin(2 * np.pi * 5 * t)) * 2 - 1`? Explain why.

Answer: The two expressions give the same value at 999 of the 1,000 samples. Only the first sample differs: at $t = 0$, `np.sign()` gives 0, while the replacement gives -1.

For a positive sine value, rounding up gives 1, so the replacement gives $+1$. For a negative value strictly between -1 and 0, rounding up gives 0, so the result is -1. At zero, it also gives -1, while `np.sign(0)` gives 0. If the sine is exactly -1, the replacement gives $2(-1) - 1 = -3$. No sampled sine value was exactly -1, so -3 did not appear in these results.

Q4. Create a composite signal by adding the sine wave and square wave together. Plot this new signal and describe its characteristics.

Answer: The original 10 Hz sine was added to the 5 Hz square. The sine gives the curved parts, and the square gives sudden jumps. The sampled values range from about -2 to $+2$ (Figure 3).

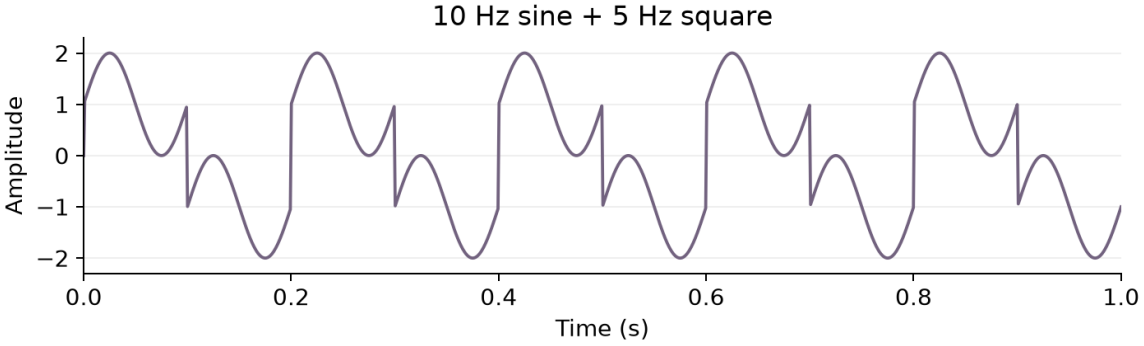


Figure 3: The original sine and square added together, without changing their amplitudes.

Q5. How might understanding different signal types be useful in real-world applications? Provide an example.

Answer: A square wave can act as a clock signal in a digital circuit. Its regular changes tell parts of the circuit when to act. Its frequency tells us how often these changes repeat.

3.2 Fourier Transform and Frequency Domain Analysis

Q1. What frequencies are present in the signal? How can you identify them from the frequency domain plot?

Answer: The signal contains 10 Hz and 20 Hz. They give the two largest FFT peaks, at 9.99 and 19.98 Hz (Figure 4). Using the actual time step gives FFT bins 0.999 Hz apart. The peaks are at the nearest bins to the input frequencies.

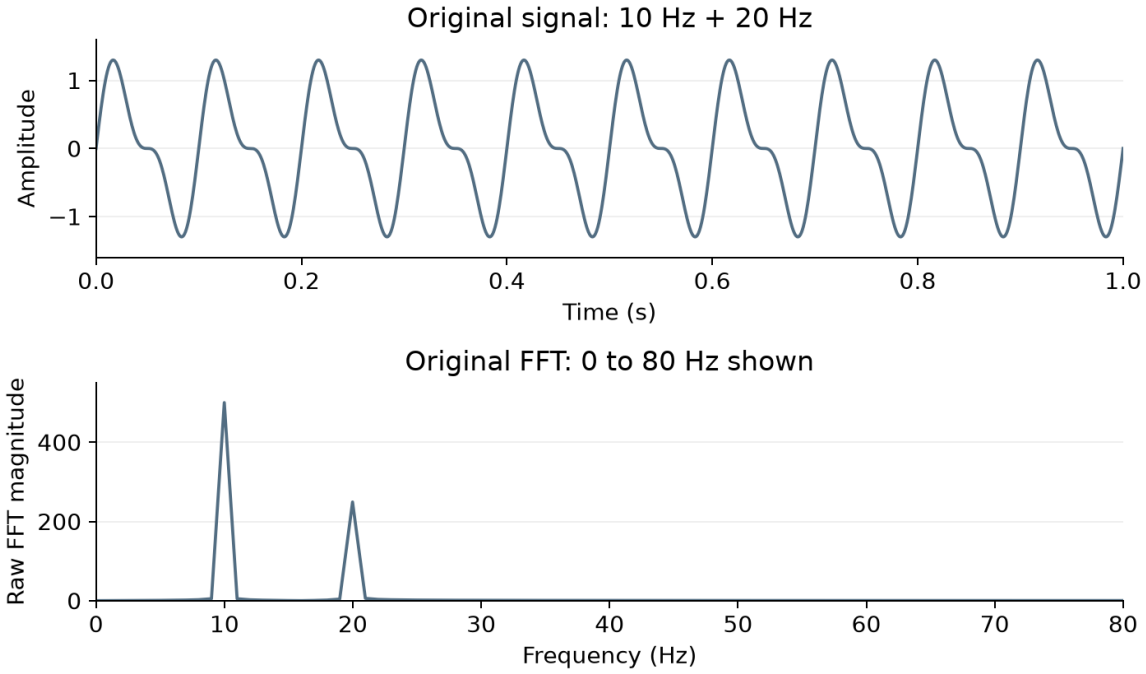


Figure 4: The original two-sine signal and its FFT magnitude. The frequency plot shows 0–80 Hz.

Q2. Modify the generate_signal function to include a 30 Hz component. How does this change the frequency domain representation?

Answer: A 30 Hz sine with amplitude 0.3 was added. A new peak appears at 29.97 Hz, with a raw FFT magnitude of 148.72. The 10 Hz and 20 Hz peaks remain (Figure 5).

Q3. What happens to the frequency domain plot if you add random noise to the signal? Implement this and explain your observations.

Answer: Gaussian noise with standard deviation 0.18 and seed 1729 was added to the original signal. Small, irregular values appear across the spectrum, while the two largest peaks stay at 9.99 and 19.98 Hz. This happens because the noise contains many frequencies (Figure 5).

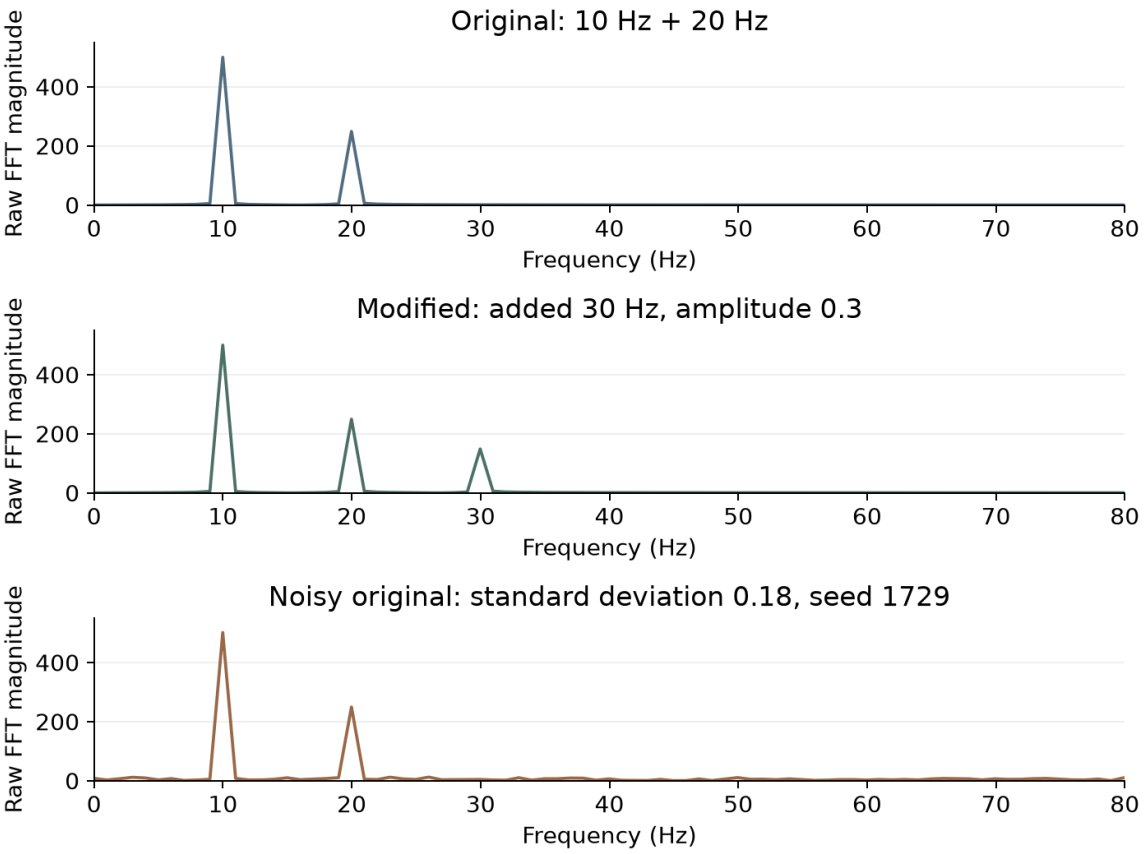


Figure 5: Original, modified and noisy spectra, using the same frequency and magnitude limits. Noise was added to the original signal.

Q4. Create a function that generates a signal with a frequency that changes over time (e.g., a chirp signal). Plot this signal in both time and frequency domains and explain what you observe.

Answer: The function sweeps from 15 Hz to 65 Hz in one second. The waves get closer together over time. The FFT shows a broad band around this range instead of two narrow peaks. It shows which frequencies are present, but not when each one occurs (Figure 6).

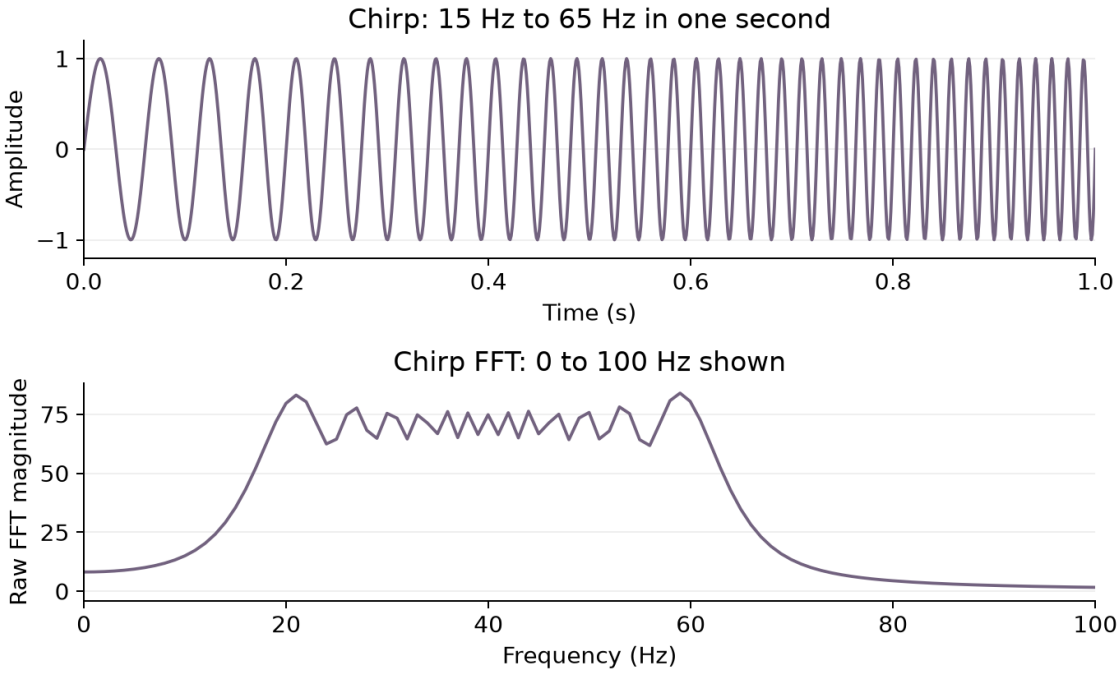


Figure 6: The chirp speeds up during the recording. Its FFT magnitude is shown over 0–100 Hz.

Q5. Describe a practical application where understanding the frequency domain of a signal is important.

Answer: A sensor can measure a machine's vibration. Its frequency peaks can be compared with a normal reading. New or much stronger peaks can help find a fault.

3.3 Sampling and Aliasing

Q1. What is the Nyquist rate for the given signal? Is the current sampling rate sufficient?

Answer: For a 10 Hz wave, the Nyquist rate is 20 Hz. The given 15 Hz sampling rate is too low, so the sampled signal shows a false 5 Hz pattern (Figure 7).

Q2. Increase the sampling rate to 25 Hz. How does this affect the sampled signal's representation of the original signal?

Answer: I increased the sampling rate to 25 Hz. This is above the 20 Hz Nyquist rate, so there is no aliasing. The extra sample points represent the original 10 Hz wave better (Figure 7).

Q3. Set the signal frequency to 20 Hz and the sampling rate to 30 Hz. Describe and explain the resulting aliasing effect.

Answer: I sampled a 20 Hz sine wave at 30 Hz. This is below the required 40 Hz Nyquist rate. The sampled points behave like a -10 Hz sine wave because the sampling rate is too low (Figure 7).

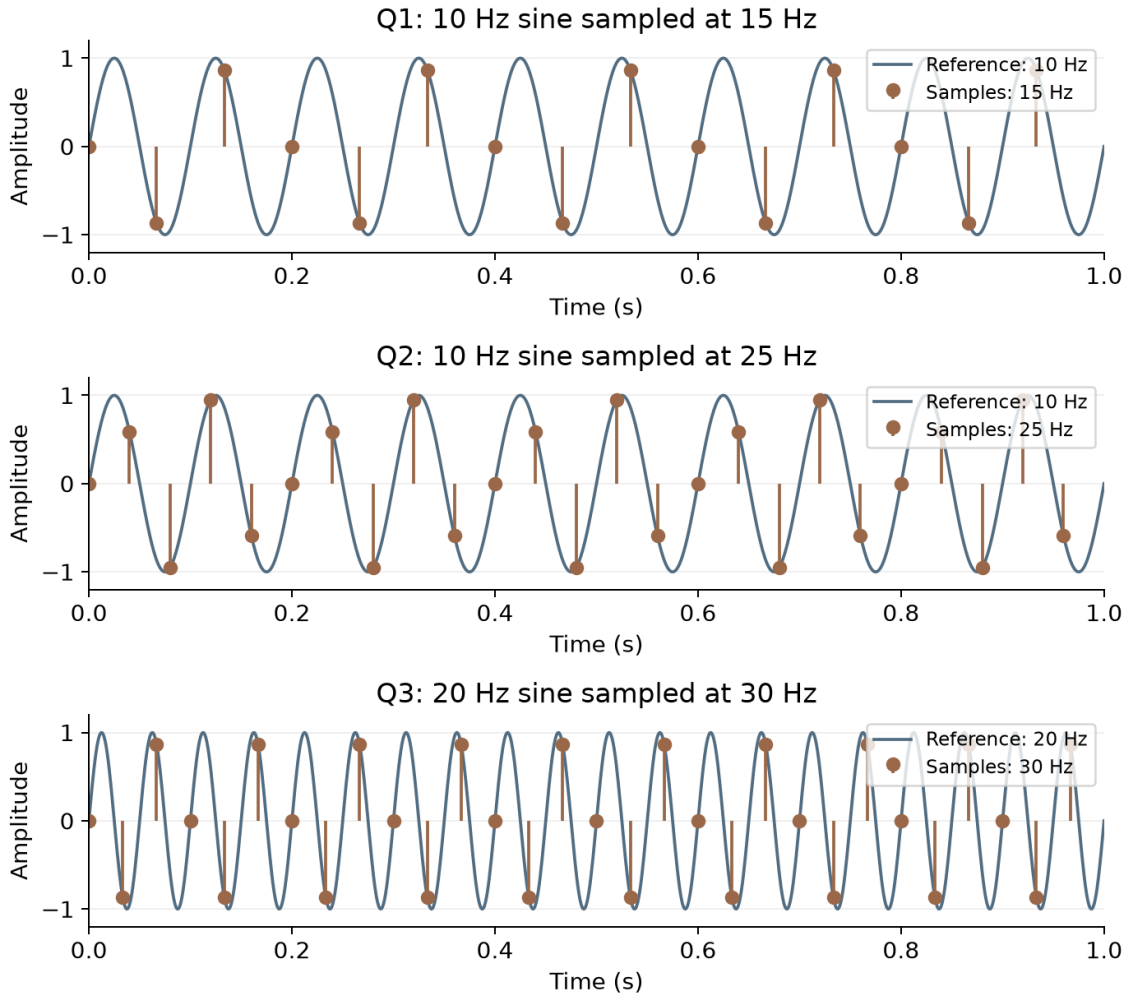


Figure 7: Sampling comparison for the three given cases. The 25 Hz sampling rate represents the 10 Hz signal without aliasing.

Q4. Create a function that demonstrates the "wagon-wheel effect" by sampling a high-frequency sinusoid at a much lower rate. Plot the original and sampled signals, and explain the visual effect.

Answer: I sampled a 23 Hz sine wave at 8 Hz. The sampled points match a 1 Hz wave with its sign reversed. This makes a fast repeating signal look much slower, like the wagon-wheel effect (Figure 8).

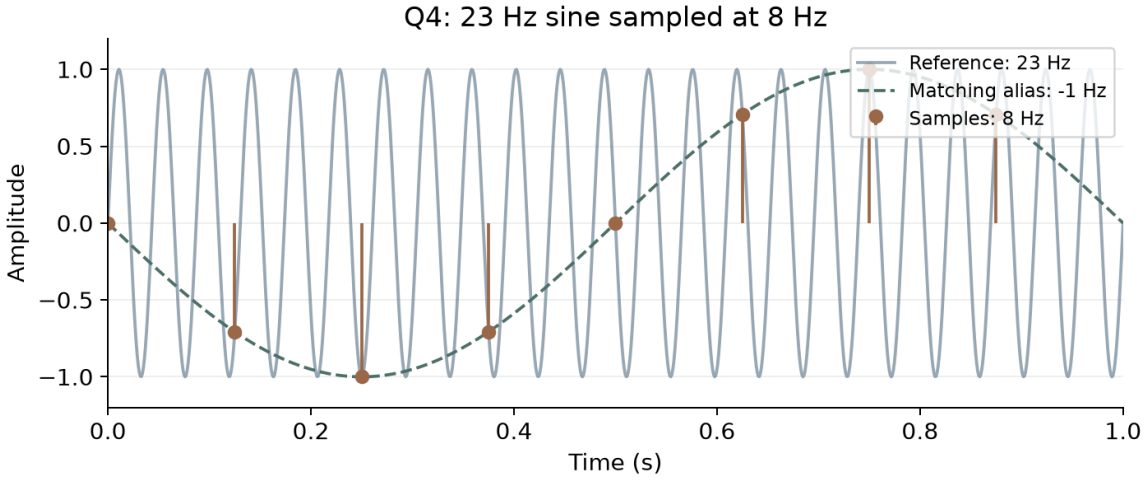


Figure 8: A 23 Hz signal sampled at 8 Hz produces a slow, reversed 1 Hz alias.

Q5. Describe a real-world scenario where understanding aliasing is crucial.

Answer: A camera can make spinning fan blades look slower or move backwards when the frame rate is too low. Choosing a high enough frame rate helps show the motion more accurately.

3.4 Image Sampling and Quantization

Q1. How does the resampling factor affect the image quality? At what factor does the distortion become apparent?

Answer: I tested resampling factors 2, 4 and 8. Factor 2 kept most details, while factor 4 made the image clearly softer and more blocky. At factor 8, the loss of detail was strong, so the distortion was easy to see.

Q2. What is the characteristic of the distortion introduced by resampling? How does it differ from the distortion introduced by requantization?

Answer: Resampling removes spatial detail and can make edges look blocky or jagged. Requantization keeps the same image size but reduces the number of gray levels, so smooth areas show banding or contour steps. The two distortions are different: one removes detail, and the other removes tone levels.

Q3. How few bits per pixel can be used in requantization before the image distortion becomes apparent?

Answer: In my tests, 4 bits per pixel was about the point where distortion became clearly visible. At 6 bits the image still looked close to the original, but at 2 bits the banding was strong. So 4 bpp was the lowest level I could use before the loss became obvious.



Figure 9: Grayscale Lenna: original, simple resampling (factor 4), and requantization (4 bpp).

Q4. Implement a function that applies both resampling and requantization to the image. Experiment with different combinations of factors and bits per pixel. What combination gives the best balance between compression and visual quality?

Answer: I combined both methods and compared several settings. The best balance was resampling by a factor of 2 with 4 bits per pixel. It reduced data a lot but still kept the main shapes and most important details.



Figure 10: Combined resampling and requantization of Lenna at the four requested settings.

Q5. Modify the code to implement a more advanced resampling method, such as bicubic interpolation. Compare the results with the simple resampling method used initially. Discuss the trade-offs between image quality and computational complexity.

Answer: I also tried bicubic resampling. It looked smoother and less blocky than the simple method, especially around edges, but it needed more computation. The simple method is faster, while bicubic gives better image quality.

Q6. Try using different images by changing the image_url. You can find a variety of test images at the USC-SIPI Image Database (`http://sipi.usc.edu/database/`). How do the effects of resampling and requantization vary for different types of images (e.g., portraits, textures, aerial photos)?

Answer: The portrait image kept its main shapes for longer, especially under requantization. The texture image lost fine detail sooner when I resampled it. Smooth regions showed banding more clearly, while detailed regions mainly lost texture.

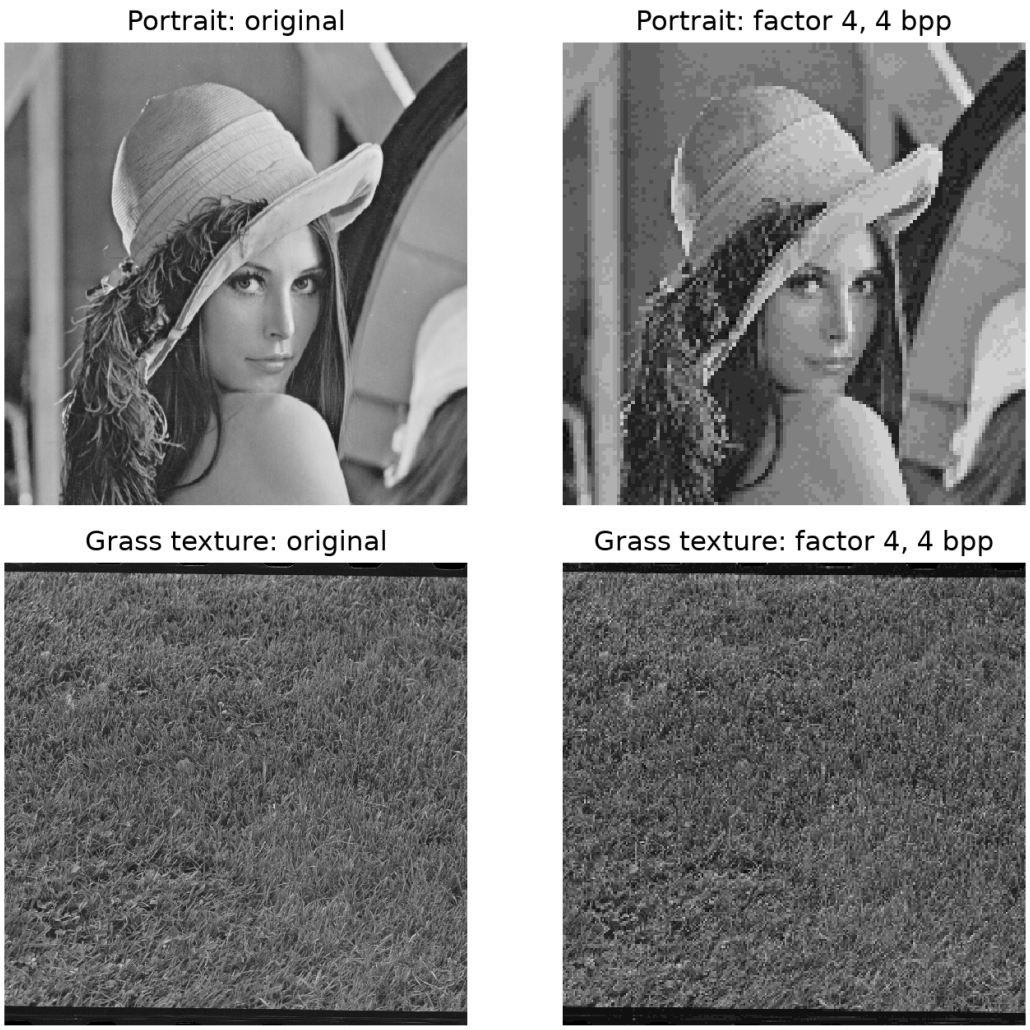


Figure 11: Portrait and grass texture before and after factor 4 resampling and 4 bpp requantization.

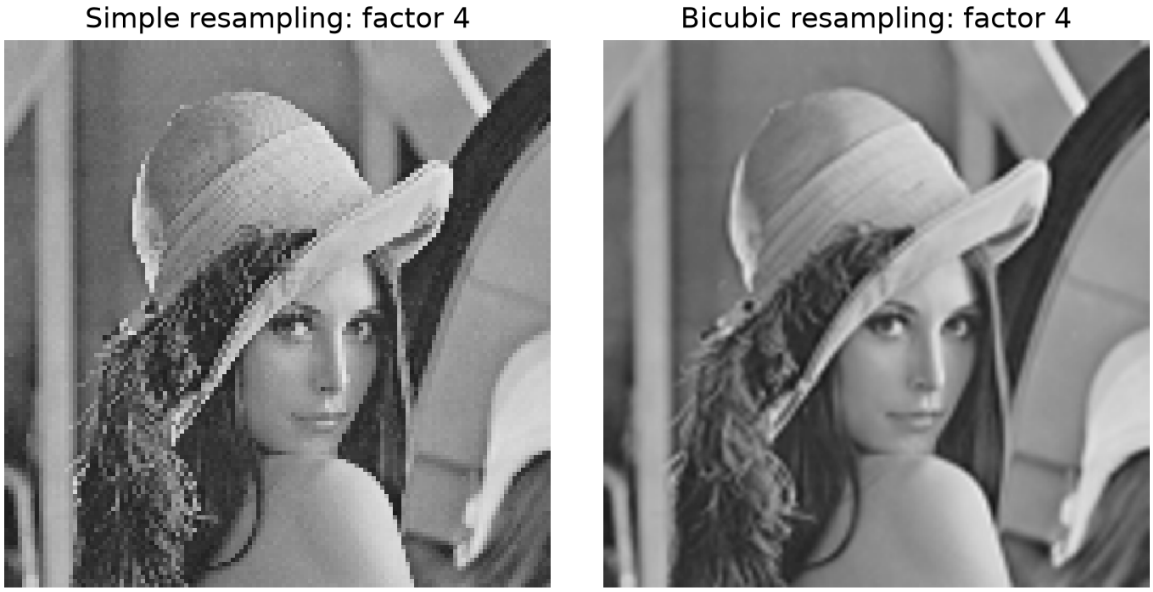


Figure 12: Simple and bicubic resampling of Lenna at factor 4.

4 Discussion

Changing frequency changes the number of cycles, while adding signals changes their shape. The square-wave comparison also shows why values at zero need to be checked. With added noise, the main FFT peaks stayed visible above the smaller values. When the sampling rate was too low, higher frequencies appeared as lower ones. Resampling changed image detail, while requantization changed the gray levels.

5 Conclusion

The requested signals were generated and compared. The 20 Hz sine had twice as many cycles, and the composite had both curves and jumps. Fourier analysis identified the added 30 Hz component and the chirp's wider frequency range. The sampling experiments showed how aliasing can give a false impression of the original signal. The image comparisons showed different losses of detail and tone levels.

Appendix: Code Snippets

```
sawtooth_wave = 2 * (5 * t - np.floor(5 * t)) - 1
square_wave_ceil = np.ceil(np.sin(2 * np.pi * 5 * t)) * 2 - 1
```

The first line removes whole cycles to make a repeating ramp. The second line is the teacher's replacement expression checked in Q3. The full script is `code/section1_1.py`. The Fourier analysis script is `code/section1_2.py`.

Sources: DT081A Lab 1 assignment, Sections 1.1–1.2 (pages 1–2), and the DT081A lab report template.